Assignment No 9

Aim: Apply object detection from CCTV Footage.

Theory:

Object Detection is a computer vision technique used to **identify and locate objects present in an image or video**. Unlike image classification, which only determines what an image contains, object detection also determines **where the object is located** by drawing a bounding box around it.

In CCTV footage, object detection can be used to automatically identify objects such as **people, vehicles, bicycles, bags, and other relevant objects** from video frames. This makes it useful for security, surveillance, traffic monitoring, and other real-world applications.

Object Detection in CCTV

A CCTV camera continuously captures video in the form of a sequence of frames. Each frame can be processed by an object detection model. The model identifies objects in the frame and provides:

- **Object class** – what the object is.
- **Bounding box** – where the object is located.
- **Confidence score** – how confident the model is about its prediction.

For example, if a CCTV frame contains two people and one car, the system may detect:

Person → Confidence: 0.92

Person → Confidence: 0.87

Car → Confidence: 0.95

Bounding boxes are then drawn around these objects.

Object Detection Models

Several deep learning models can be used for object detection:

1. **YOLO (You Only Look Once):**
   YOLO is a popular real-time object detection algorithm. It processes an image in a single pass through the neural network, making it suitable for CCTV and real-time applications.
2. **SSD (Single Shot Detector):**
   SSD is another single-stage object detection method that provides a balance between speed and accuracy.
3. **Faster R-CNN:**
   Faster R-CNN is a two-stage object detection model that generally provides high

detection accuracy but can be slower than models designed specifically for real-time detection.

YOLO for CCTV Object Detection

For a practical implementation, **YOLO** can be used to detect objects from a CCTV video.

The general process is:

CCTV Video

↓

Extract Video Frames

↓

Object Detection Model

↓

Identify Objects

↓

Draw Bounding Boxes

↓

Display/Save Detected Video

Steps Involved

1. **Input CCTV footage:**
   Load a recorded CCTV video using a video-processing library such as OpenCV.
2. **Read video frames:**
   The video is divided into individual frames.
3. **Load the object detection model:**
   A pretrained model such as YOLO is loaded.
4. **Process each frame:**
   Each frame is passed to the detection model.
5. **Detect objects:**
   The model identifies objects present in the frame.
6. **Calculate confidence:**
   A confidence score is generated for every detected object.
7. **Apply threshold:**
   Detections below a specified confidence threshold can be ignored.

8. **Draw bounding boxes:**
   Rectangles are drawn around detected objects along with their class names and confidence scores.
9. **Display the result:**
   The processed frames are displayed as a video.
10. **Save the output:**
    The detected video can optionally be saved for further analysis.

Important Terms

Bounding Box:
A rectangular box drawn around a detected object.

Confidence Score:
A numerical value indicating how confident the model is that a particular object has been detected correctly.

Class Label:
The name of the detected object, such as person, car, bicycle, or bus.

IoU (Intersection over Union):
A metric used to measure the overlap between the predicted bounding box and the actual bounding box.

$$IoU = \frac{Area\ of\ Intersection}{Area\ of\ Union}$$

Confidence Threshold:
A minimum confidence value used to filter out weak or uncertain detections.

Applications

CCTV object detection can be used for:

- **Security and surveillance**
- **Intrusion detection**
- **Vehicle detection**
- **Traffic monitoring**
- **Crowd monitoring**
- **Parking management**
- **People counting**
- **Suspicious activity monitoring**
- **Industrial safety monitoring**

- **Restricted-area monitoring**

Advantages

- Enables automated CCTV monitoring.
- Can process video continuously.
- Reduces the need for constant manual observation.
- Can detect multiple objects in a single frame.
- Modern models can work in real time.
- Useful for security and traffic applications.

Limitations

- Detection accuracy can be affected by poor lighting.
- Crowded scenes may make object detection difficult.
- Objects that are partially hidden may not be detected correctly.
- Camera angle and video quality can affect performance.
- Real-time detection may require significant computational resources.

Conclusion

Object detection from CCTV footage uses **computer vision and deep learning** to automatically identify and locate objects in video frames. Models such as **YOLO** can detect multiple objects and display their class labels, confidence scores, and bounding boxes. This technology is widely used in surveillance, security, traffic monitoring, and automated video analysis.

Code:

```
import cv2
from ultralytics import YOLO


# Load the pre-trained YOLO model
model = YOLO("yolo11n.pt")


# Open the CCTV video
cap = cv2.VideoCapture("crosswalk.avi")
```

```
while cap.isOpened():

    # Read one frame
    ret, frame = cap.read()

    # Stop when video ends
    if not ret:
        break

    # Detect objects
    results = model(frame)

    # Draw bounding boxes
    annotated_frame = results[0].plot()

    # Display the video
    cv2.imshow("CCTV Object Detection", annotated_frame)

    # Press Q to stop
    if cv2.waitKey(1) & 0xFF == ord("q"):
        break

# Release video
cap.release()
cv2.destroyAllWindows()
```

Output:




```
Problems  Output  Debug Console  Terminal  Ports                                                    powershell

0: 384x640 1 person, 73.7ms
Speed: 5.3ms preprocess, 73.7ms inference, 1.3ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 1 person, 1 skateboard, 55.1ms
Speed: 2.5ms preprocess, 55.1ms inference, 1.5ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 1 person, 56.6ms
Speed: 3.3ms preprocess, 56.6ms inference, 1.0ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 1 person, 69.2ms
Speed: 3.0ms preprocess, 69.2ms inference, 1.7ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 1 person, 68.1ms
Speed: 4.7ms preprocess, 68.1ms inference, 1.3ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 1 person, 63.6ms
Speed: 2.3ms preprocess, 63.6ms inference, 1.3ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 1 person, 62.0ms
Speed: 2.5ms preprocess, 62.0ms inference, 1.1ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 1 person, 1 skateboard, 125.3ms
Speed: 3.0ms preprocess, 125.3ms inference, 1.7ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 1 person, 68.8ms
Speed: 5.0ms preprocess, 68.8ms inference, 1.7ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 1 person, 71.5ms
Speed: 2.5ms preprocess, 71.5ms inference, 1.5ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 1 person, 72.3ms
Speed: 2.8ms preprocess, 72.3ms inference, 1.4ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 1 person, 71.0ms
Speed: 3.0ms preprocess, 71.0ms inference, 1.3ms postprocess per image at shape (1, 3, 384, 640)
```